

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

Nghiên cứu tốt nghiệp 1

Mô hình nhận dạng cảm xúc khuôn mặt

Lê Nguyễn Minh Đức

Duc.LNM225611@sis.hust.edu.vn

Ngành: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Nguyễn Thị Hoàng Lan

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2025

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Giới thiệu Đề Tài	1
1.1.1 Tương tác người - máy.....	1
1.1.2 Chăm sóc sức khỏe tâm thần và y tế.....	1
1.1.3 Marketing và nghiên cứu hành vi.....	1
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	2
2.1 Ngữ cảnh của bài toán.....	2
2.2 Thuật toán phát hiện khuôn mặt	2
2.2.1 Phương Pháp Haar Cascade.....	2
2.2.2 Multi-task Cascaded Convolutional Networks (MTCNN)	5
2.3 Mô hình ResNet	7
2.3.1 Những Thách Thức Mà Mạng Nơ-Ron Sâu Gặp Phải	7
2.3.2 Các Thành Phần Chính Trong Kiến Trúc ResNet	8
2.3.3 Tổng Kết	9
2.4 Học chuyển giao (Transfer Learning).....	9
2.4.1 Cơ chế hoạt động	9
2.4.2 Cấu trúc mô hình Transfer Learning	10
2.4.3 Quy trình huấn luyện cơ bản	10
2.4.4 Ưu điểm của Transfer Learning	11
CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM	12
3.1 Tổng quan mô hình	12
3.2 Dữ liệu sử dụng	12
3.3 Pipeline chi tiết.....	13

3.4 Tiến hành triển khai thực nghiệm	13
3.4.1 Tiền xử lý dữ liệu	13
3.4.2 Fine-tune trên ResNet18:	15
3.4.3 Đánh giá	20
3.4.4 Triển khai.....	26
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	32
4.1 Kết luận	32
4.2 Hướng phát triển trong tương lai	32

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu Đề Tài

Cảm xúc đóng vai trò vô cùng quan trọng trong cuộc sống hàng ngày của chúng ta, giúp chúng ta hiểu và phản ứng phù hợp với môi trường xung quanh. Nhận diện cảm xúc là một khả năng cơ bản của con người, và trong bối cảnh hiện đại, với sự phát triển mạnh mẽ của trí tuệ nhân tạo (AI), khả năng máy tính nhận biết cảm xúc qua khuôn mặt con người đã mở ra nhiều cơ hội ứng dụng đột phá.

Trong các hệ thống người-máy, nhận diện cảm xúc qua khuôn mặt không chỉ giúp tạo ra một trải nghiệm tương tác tự nhiên mà còn đóng góp vào sự phát triển của các công nghệ tiên tiến trong nhiều lĩnh vực khác nhau, bao gồm:

1.1.1 Tương tác người - máy

Việc nhận diện cảm xúc giúp các thiết bị như trợ lý ảo, robot, và chatbot có thể nhận biết trạng thái cảm xúc của người dùng, từ đó điều chỉnh cách phản hồi sao cho phù hợp, mang đến một trải nghiệm người dùng tự nhiên và thân thiện hơn. Các ứng dụng này có thể được triển khai trong các trợ lý ảo, các hệ thống hỗ trợ khách hàng, và nhiều ứng dụng khác.

1.1.2 Chăm sóc sức khỏe tâm thần và y tế

Việc nhận diện cảm xúc có thể hỗ trợ trong việc theo dõi và phát hiện sớm các dấu hiệu của các vấn đề tâm lý như trầm cảm, lo âu, hay căng thẳng thông qua các biểu hiện cảm xúc trên khuôn mặt của bệnh nhân. Điều này không chỉ giúp phát hiện sớm các triệu chứng mà còn có thể cải thiện hiệu quả của quá trình điều trị, giúp bác sĩ và nhà tâm lý học đánh giá trạng thái cảm xúc của bệnh nhân trong thời gian thực, từ đó điều chỉnh phương pháp điều trị một cách hợp lý.

1.1.3 Marketing và nghiên cứu hành vi

Trong lĩnh vực marketing, nhận diện cảm xúc giúp phân tích cảm xúc của khách hàng khi xem quảng cáo, trải nghiệm sản phẩm, hoặc dịch vụ. Các công ty có thể tối ưu chiến dịch marketing của mình dựa trên những phản ứng cảm xúc thực tế của người tiêu dùng, nâng cao trải nghiệm người dùng và tăng cường hiệu quả tiếp thị. Đây là một công cụ mạnh mẽ trong nghiên cứu hành vi người tiêu dùng, giúp doanh nghiệp hiểu rõ hơn về khách hàng của mình.

Với những tiềm năng to lớn như vậy, nghiên cứu về nhận diện cảm xúc qua khuôn mặt không chỉ là một thách thức kỹ thuật mà còn mang lại nhiều cơ hội ứng dụng đột phá trong các lĩnh vực liên quan đến AI và thị giác máy tính.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Ngữ cảnh của bài toán

Bài toán nhận dạng cảm xúc thuộc về bài toán phân loại với đầu vào là ảnh có gương mặt và đầu ra là cảm xúc phán đoán.

Phát triển một hệ thống có thể xác định và phân loại các cảm xúc của con người thông qua các tín hiệu như hình ảnh, âm thanh, hoặc dữ liệu sinh lý. Trong lĩnh vực nhận dạng cảm xúc qua hình ảnh, bài toán này tập trung vào việc phân tích các đặc trưng khuôn mặt và các cử chỉ thể hiện cảm xúc.

Cụ thể, bài toán nhận dạng cảm xúc qua hình ảnh sử dụng các kỹ thuật học máy, đặc biệt là học sâu, để phân tích các đặc điểm khuôn mặt (ví dụ: mắt, miệng, mũi) và xác định cảm xúc như vui vẻ, buồn bã, tức giận, sợ hãi, ngạc nhiên, và ghê tởm.

Các bước trong bài toán nhận dạng cảm xúc bao gồm:

- **Phát hiện khuôn mặt:** Sử dụng các phương pháp như Haar Cascade và MTCNN để phát hiện khuôn mặt.
- **Tiền xử lý ảnh:** Cắt xén khuôn mặt, chia tập dữ liệu thành train, test, validation. Load dữ liệu vào data loader với các chỉnh sửa, thay đổi ảnh: ngẫu nhiên lật, xoay, chuyển màu, cắt xén, che đi một phần ảnh.
- **Fine-tuning với ResNet18:** Sử dụng mô hình ResNet18 đã được huấn luyện trước để fine-tune trên dữ liệu cụ thể của bài toán nhận dạng cảm xúc.
- **Đánh giá mô hình:** Thử nghiệm trên tập test, ngoài ra còn thử nghiệm trên 2 bộ dữ liệu KDEF và RAF-DB.

2.2 Thuật toán phát hiện khuôn mặt

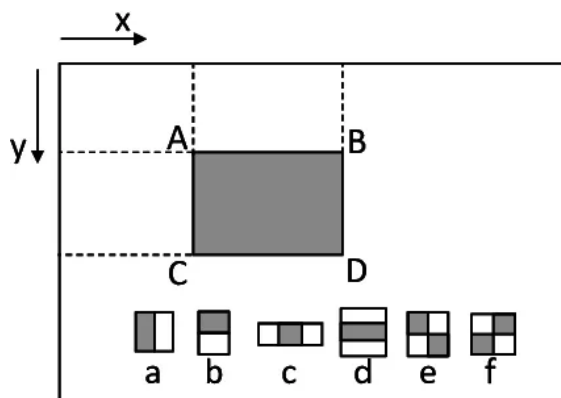
2.2.1 Phương Pháp Haar Cascade

Haar Cascade là một thuật toán cổ điển nhưng vẫn được sử dụng rộng rãi trong các ứng dụng nhận diện đối tượng, đặc biệt là nhận diện khuôn mặt. Phương pháp này dựa trên các đặc trưng Haar (Haar-like features), vốn là các chỉ số mô tả sự thay đổi cường độ ánh sáng giữa các vùng hình chữ nhật trong ảnh. Các đặc trưng này có thể được tính toán rất nhanh chóng và hiệu quả, điều này cho phép Haar Cascade có thể phát hiện đối tượng trong ảnh với độ phân giải cao mà không gặp phải vấn đề về tốc độ xử lý.

Thuật toán Haar Cascade hoạt động theo quy trình sau:

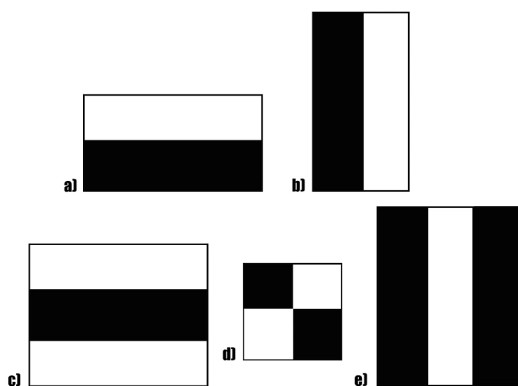
- **Quét toàn bộ hình ảnh:** Trong bước đầu tiên, thuật toán di chuyển một cửa

số nhỏ qua toàn bộ ảnh đầu vào. Mỗi cửa sổ sẽ được kiểm tra để xác định xem nó có chứa đối tượng (khuôn mặt) hay không. Quá trình này giúp nhận diện các khu vực khả nghi có thể chứa khuôn mặt mà không cần phải quét từng vùng ảnh riêng biệt.



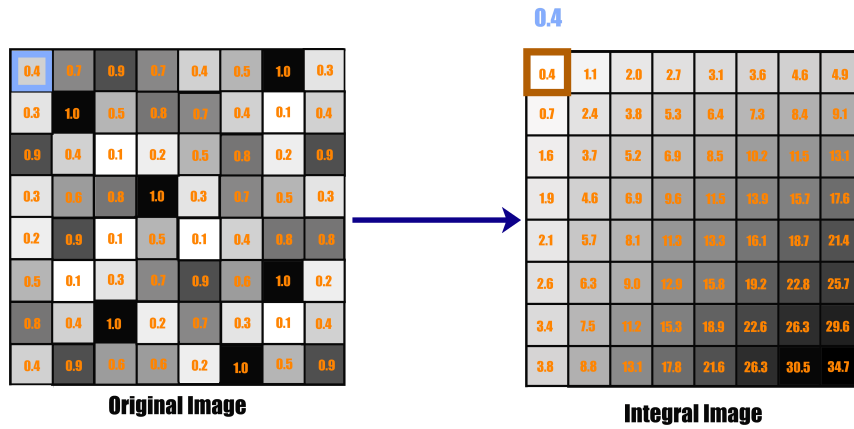
Hình 2.1: Quá trình quét của Haar Cascade qua ảnh

- **Tính toán các đặc trưng Haar:** Các đặc trưng Haar được tính toán dựa trên sự khác biệt về cường độ ánh sáng giữa các vùng hình chữ nhật trong ảnh. Mỗi đặc trưng Haar cho phép đánh giá sự thay đổi về mức độ sáng tối giữa các vùng ảnh khác nhau, từ đó nhận diện được các đặc điểm quan trọng trong khuôn mặt như mắt, mũi, miệng và đường viền khuôn mặt.



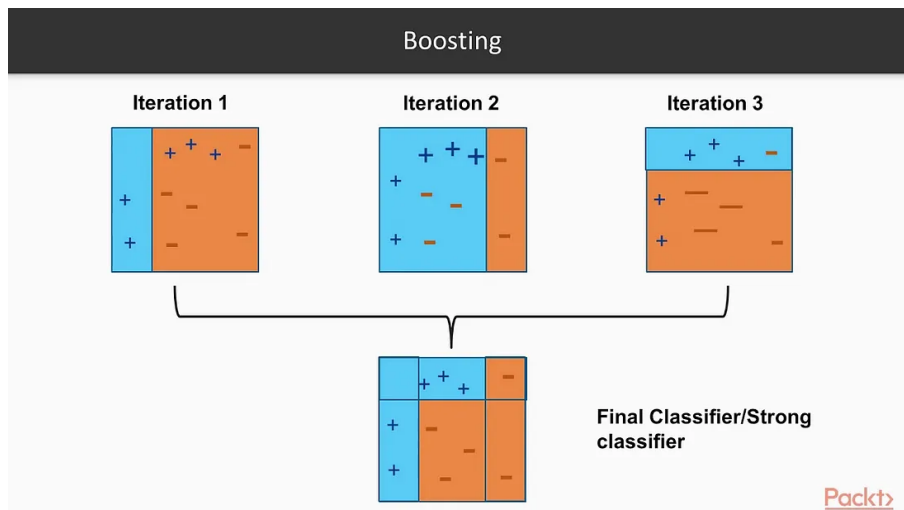
Hình 2.2: Các đặc trưng Haar được tính toán trong thuật toán

- **Tạo ảnh tích phân (Integral Image):** Một kỹ thuật quan trọng giúp tăng tốc tính toán các đặc trưng Haar là việc sử dụng ảnh tích phân. Ảnh tích phân là một phép toán cho phép tính nhanh tổng cường độ ánh sáng trong bất kỳ vùng hình chữ nhật nào của ảnh. Bằng cách này, thuật toán có thể tính toán đặc trưng Haar một cách nhanh chóng mà không cần phải tính lại tổng các điểm ảnh cho mỗi cửa sổ.



Hình 2.3: Hoạt động của ảnh tích phân giúp tính toán nhanh chóng

- **Sử dụng AdaBoost:** AdaBoost là một thuật toán học máy mạnh mẽ được sử dụng để chọn lọc các đặc trưng Haar quan trọng từ một tập hợp lớn các đặc trưng. Thông qua quá trình học có giám sát, AdaBoost kết hợp các đặc trưng yếu để tạo thành một mô hình phân loại mạnh mẽ, từ đó giúp thuật toán Haar Cascade đạt được độ chính xác cao trong việc phát hiện khuôn mặt. AdaBoost giúp giảm thiểu số lượng đặc trưng cần thiết mà vẫn duy trì được hiệu quả trong quá trình nhận diện.



Hình 2.4: Quá trình hoạt động của AdaBoost trong Haar Cascade

Tổng kết: Haar Cascade là một phương pháp hiệu quả và nhanh chóng trong việc phát hiện khuôn mặt, đặc biệt khi yêu cầu xử lý với độ phân giải cao và trong các điều kiện ánh sáng không đồng đều. Với khả năng tính toán nhanh chóng thông qua các đặc trưng Haar và ảnh tích phân, cộng với khả năng chọn lọc đặc trưng mạnh mẽ từ AdaBoost, Haar Cascade không chỉ giảm thiểu độ phức tạp tính toán mà còn mang lại độ chính xác cao trong việc nhận diện khuôn mặt trong các bối

cảnh thực tế. Mặc dù có một số hạn chế trong việc nhận diện khuôn mặt ở góc độ nghiêng hoặc ảnh bị che khuất, Haar Cascade vẫn là một lựa chọn phổ biến nhờ vào tính hiệu quả và đơn giản của nó.

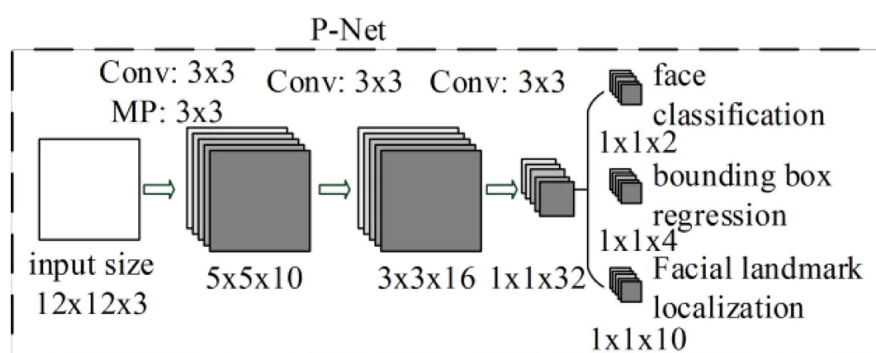
2.2.2 Multi-task Cascaded Convolutional Networks (MTCNN)

MTCNN là một phương pháp hiện đại, phát hiện khuôn mặt và căn chỉnh các đặc điểm của khuôn mặt (facial landmark alignment). MTCNN gồm ba mạng CNN kết hợp để nhận diện khuôn mặt và xác định vị trí các đặc điểm trên khuôn mặt, đồng thời thực hiện nhiệm vụ phát hiện khuôn mặt, hồi quy bounding box và xác định các đặc điểm khuôn mặt.

a, Ba giai đoạn của MTCNN

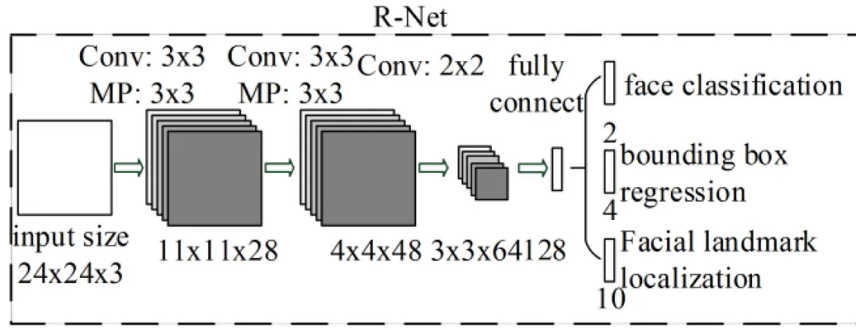
MTCNN thực hiện ba giai đoạn chính với sự kết hợp của các mạng CNN, mỗi giai đoạn giải quyết một nhiệm vụ cụ thể:

1. **Giai đoạn 1: Proposal Network (P-Net):** P-Net là một mạng hoàn toàn tích chập (Fully Convolutional Network - FCN) và được sử dụng để tạo ra các cửa sổ đề xuất (candidate windows) cho khuôn mặt và thực hiện hồi quy bounding box. Sau khi tìm được các cửa sổ đề xuất, các cửa sổ này sẽ được tinh chỉnh và loại bỏ những khu vực trùng lặp. P-Net giúp nhanh chóng tạo ra các cửa sổ chứa khuôn mặt tiềm năng.



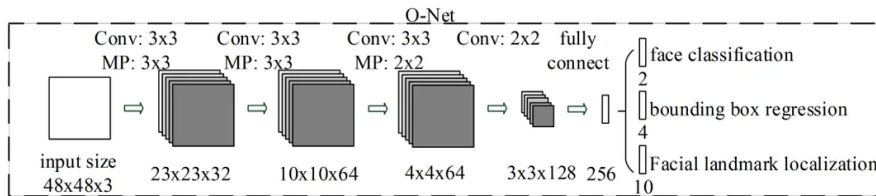
Hình 2.5: P-Net: Giai đoạn đề xuất cửa sổ

2. **Giai đoạn 2: Refine Network (R-Net):** Tất cả các cửa sổ từ P-Net được đưa vào R-Net, một mạng CNN hoàn chỉnh với các lớp dense. R-Net thực hiện việc giảm số lượng cửa sổ đề xuất, cải thiện chính xác vị trí bounding box và áp dụng phương pháp non-maximum suppression (NMS) để loại bỏ các cửa sổ trùng lặp. R-Net trả về ba kết quả: xác định khuôn mặt hay không, một vector 4 phần tử chứa thông tin bounding box, và một vector 10 phần tử cho việc định vị các đặc điểm khuôn mặt.
3. **Giai đoạn 3: Output Network (O-Net):** O-Net hoạt động tương tự như R-Net



Hình 2.6: R-Net: Giai đoạn tinh chỉnh cửa sổ

nhưng với mục tiêu chính là mô tả chi tiết khuôn mặt và xác định chính xác vị trí của năm đặc điểm khuôn mặt: mắt trái, mắt phải, mũi, góc miệng trái, và góc miệng phải. Đây là bước cuối cùng giúp hoàn thiện việc nhận diện và căn chỉnh các đặc điểm khuôn mặt.



Hình 2.7: O-Net: Giai đoạn mô tả chi tiết khuôn mặt

b. Các nhiệm vụ của MTCNN

MTCNN thực hiện ba nhiệm vụ chính sau trong quá trình huấn luyện và dự đoán:

1. **Phân loại khuôn mặt/non-khuôn mặt (Face/Non-face):** Để phân loại một vùng ảnh có chứa khuôn mặt hay không, MTCNN sử dụng hàm mất mát cross-entropy. Đây là bài toán phân loại nhị phân giữa khuôn mặt và không phải khuôn mặt.

$$L_i^{det} = -(y_i^{det} \log(P_i) + (1 - y_i^{det}) \log(1 - P_i))$$

2. **Hồi quy bounding box:** Đây là bài toán hồi quy, trong đó MTCNN học cách điều chỉnh vị trí của bounding box sao cho khớp với các khuôn mặt trong ảnh. Phương pháp này sử dụng độ đo lỗi Euclidean để tính toán sự chênh lệch giữa vị trí dự đoán và vị trí thực tế của bounding box.

$$L_i^{box} = \|\hat{y}_i^{box} - y_i^{box}\|_2^2$$

3. **Định vị các đặc điểm khuôn mặt:** MTCNN sử dụng một bài toán hồi quy để xác định chính xác vị trí của năm đặc điểm khuôn mặt: hai mắt, mũi, và hai góc miệng. Lỗi được tính bằng độ lệch Euclidean giữa các điểm dự đoán và điểm thực tế.

$$L_i^{landmark} = \|\hat{y}_i^{landmark} - y_i^{landmark}\|_2^2$$

c, Tổng kết về MTCNN

MTCNN là mô hình mạnh mẽ và hiệu quả nhờ khả năng thực hiện đồng thời ba tác vụ quan trọng: phát hiện khuôn mặt, hồi quy bounding box, và căn chỉnh các đặc điểm khuôn mặt. Các ba giai đoạn của MTCNN giúp tối ưu hóa khả năng nhận diện khuôn mặt và đặc điểm khuôn mặt trong các điều kiện khác nhau, từ đó nâng cao độ chính xác của mô hình trong các ứng dụng thực tế. MTCNN không chỉ sử dụng các mạng CNN để phát hiện khuôn mặt mà còn tích hợp các kỹ thuật học đa nhiệm (multi-task learning) để hoàn thành đồng thời các tác vụ này.

2.3 Mô hình ResNet

Mạng Residual (ResNet) là một mô hình học sâu được sử dụng rộng rãi trong các ứng dụng nhận dạng hình ảnh. Kiến trúc ResNet (Residual Neural Network) được giới thiệu bởi Kaiming He, Xiangyu Zhang, Shaoqing Ren và Jian Sun trong bài báo “Deep Residual Learning for Image Recognition” vào năm 2015.

Một trong những phát triển quan trọng nhất trong Cuộc thi ILVRSC (ImageNet Large Scale Visual Recognition Challenge) 2015 là sự ra đời của mạng Residual. ResNet đã đạt được những kết quả chưa từng có nhờ giải quyết các thách thức liên quan đến việc huấn luyện các mạng nơ-ron sâu. ResNet đã vượt trội hơn các kiến trúc khác và giành chiến thắng trong bài toán phân loại hình ảnh của ILVRSC 2015 với tỷ lệ lỗi top-5 là 3.57% và đạt được kết quả ấn tượng.

2.3.1 Những Thách Thức Mà Mạng Nơ-Ron Sâu Gặp Phải

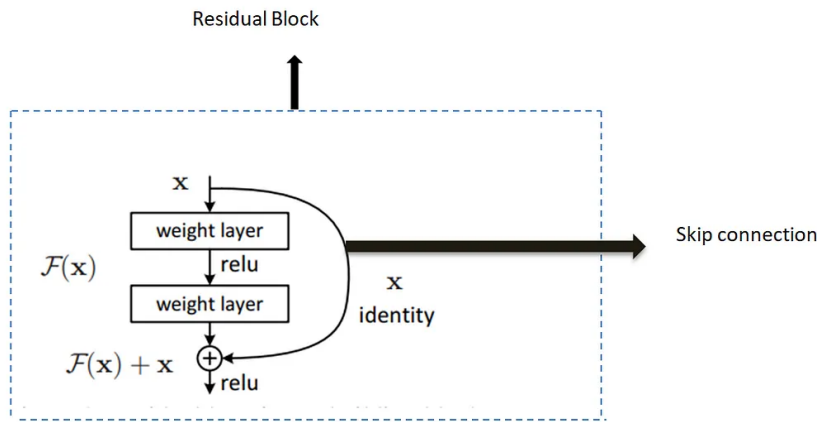
- **Vấn đề Vanishing/Exploding Gradient:** Khi số lượng lớp trong mạng nơ-ron tăng lên, gradient của hàm mất mát đối với trọng số có thể trở nên cực kỳ nhỏ trong quá trình lan truyền ngược. Điều này khiến các lớp sau không thể học được các mẫu có ý nghĩa, vì các cập nhật trọng số gần như không có tác động gì. Ngược lại, trong một số trường hợp, gradient có thể trở nên rất lớn trong quá trình lan truyền ngược, gây ra sự không ổn định trong quá trình huấn luyện và làm cho việc cập nhật trọng số trở nên không kiểm soát được.
- **Vấn Đề Suy Giảm Trong Mạng Nơ-Ron:** Vấn đề suy giảm trong mạng nơ-ron đề cập đến hiện tượng khi độ sâu của mạng nơ-ron tăng lên, hiệu suất của mạng trên dữ liệu huấn luyện bão hòa và sau đó bắt đầu suy giảm. Vấn đề này

đặc biệt quan trọng vì nó trái ngược với trực giác rằng các mạng sâu hơn có khả năng trích xuất các đặc trưng phức tạp và trừu tượng. Chúng ta có thể thấy vấn đề này dưới hai khía cạnh:

- **Bão hòa hiệu suất:** Khi số lượng lớp trong mạng nơ-ron tăng lên, lỗi huấn luyện có xu hướng bão hòa và không cải thiện nữa. Điều này có nghĩa là các lớp bổ sung không mang lại lợi ích gì đáng kể trong việc giảm lỗi huấn luyện.
- **Suy giảm độ chính xác:** Sau khi thêm nhiều lớp vào mạng, bất ngờ lỗi trên bộ dữ liệu xác thực bắt đầu tăng lên và hiệu suất trên dữ liệu chưa thấy trở nên kém đi.

2.3.2 Các Thành Phần Chính Trong Kiến Trúc ResNet

1. **Residual Block:** Các residual block là thành phần chính của mạng Residual. Trong mạng nơ-ron cổ điển, đầu vào được chuyển qua một bộ lớp chập rồi được đưa qua hàm kích hoạt. Trong mạng Residual, đầu vào của khối được cộng với đầu ra của khối, tạo ra một kết nối residual. Đầu ra của residual block $H(x)$ có thể được biểu diễn bởi:



Hình 2.8: Residual Block

$$H(x) = F(x) + x$$

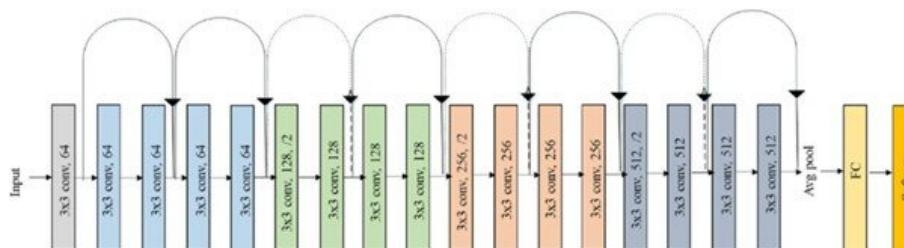
Trong đó, $F(x)$ là phép toán residual được học bởi mạng. Sự hiện diện của từ đồng nhất x giúp gradient lan truyền dễ dàng hơn.

2. **Skip Connection:** Skip connection giúp hình thành các residual block. Skip connection bao gồm việc đầu vào của residual block được bỏ qua qua lớp chập và cộng vào đầu ra của residual block.

3. **Các lớp chồng chất:** Kiến trúc ResNet được tạo thành từ việc xếp chồng nhiều residual block lại với nhau. Sử dụng các residual block này, kiến trúc ResNet có thể được xây dựng rất sâu. Các phiên bản ResNet với 50, 101, và 152 lớp đã được giới thiệu.
4. **Global Average Pooling (GAP):** Các kiến trúc ResNet thường sử dụng Global Average Pooling như lớp cuối cùng trước lớp fully connected. GAP giảm kích thước không gian xuống một giá trị duy nhất cho mỗi feature map, cung cấp một đại diện tổng quát của toàn bộ feature map.

2.3.3 Tổng Kết

ResNet là một kiến trúc mạng nơ-ron sâu đã giải quyết những vấn đề lớn trong huấn luyện các mạng rất sâu thông qua việc sử dụng các residual block và skip connection. Việc sử dụng các phương pháp này giúp mạng có thể học được các đặc trưng phức tạp và tránh được các vấn đề như biến mất gradient hay suy giảm hiệu suất khi độ sâu tăng lên. ResNet đã đạt được thành công lớn trong các cuộc thi nhận dạng hình ảnh và hiện nay là một trong những kiến trúc phổ biến trong các ứng dụng thị giác máy tính.



Hình 2.9: ResNet18

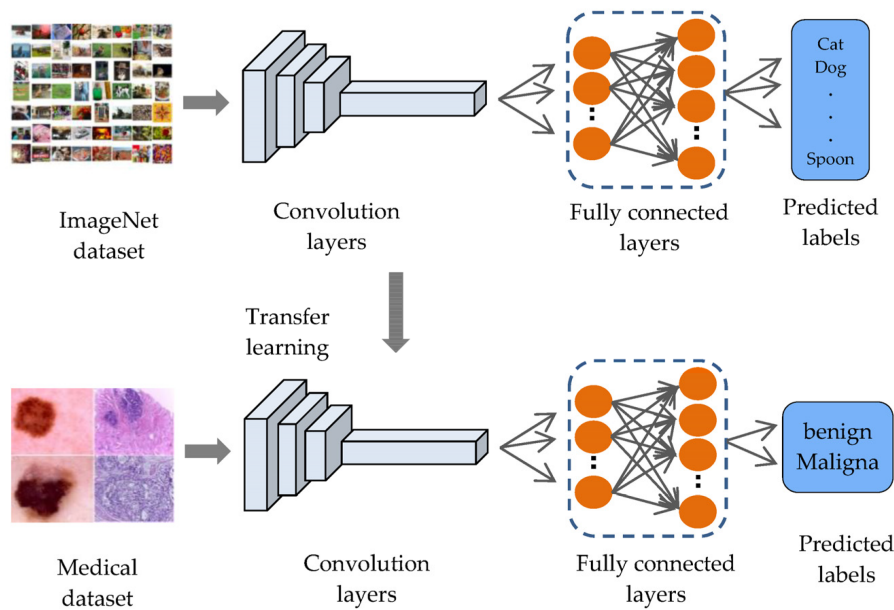
2.4 Học chuyển giao (Transfer Learning)

Học chuyển giao (Transfer Learning) là một phương pháp trong học máy, cho phép tận dụng các kiến thức từ những mô hình đã được huấn luyện trước (pretrained models) để giải quyết các bài toán mới, nhằm tiết kiệm tài nguyên tính toán và giảm thời gian huấn luyện.

2.4.1 Cơ chế hoạt động

Trong Transfer Learning, các mạng CNN hoạt động như một công cụ trích xuất đặc trưng, với các tầng Convolutional được thiết kế để nhận diện đặc trưng ở nhiều mức độ khác nhau. Các tầng đầu tiên của mạng học được các đặc trưng bậc thấp, chẳng hạn như các đường nét cơ bản (ngang, dọc, cạnh của ảnh), trong khi các tầng cuối cùng học các đặc trưng bậc cao hơn, tổng hợp từ các đặc trưng bậc thấp, nhằm tạo ra những đặc trưng có khả năng phân loại mạnh mẽ hơn.

2.4.2 Cấu trúc mô hình Transfer Learning



Hình 2.10: Transfer Learning

Mô hình Transfer Learning bao gồm hai thành phần chính:

- **Base Network:** Thành phần này được trích xuất từ một mô hình đã huấn luyện sẵn (pretrained model), sau khi loại bỏ các lớp fully connected ở trên cùng. Nó có nhiệm vụ trích xuất các đặc trưng từ dữ liệu đầu vào.
- **Fully Connected Layers:** Các lớp này được thêm vào sau Base Network để giảm chiều dữ liệu và tính toán phân phối xác suất cho các lớp của bài toán mới, với số lượng units tại output tương ứng với số lượng các lớp trong bài toán hiện tại.

2.4.3 Quy trình huấn luyện cơ bản

- **Lựa chọn mô hình pre-trained:** Mô hình pre-trained phù hợp cần được lựa chọn dựa trên mục tiêu bài toán, loại dữ liệu và mức độ phức tạp của vấn đề.
- **Đóng băng các lớp cũ:** Sau khi lựa chọn mô hình pre-trained, bước tiếp theo là đóng băng các lớp trước đó để giữ lại những kiến thức đã học được. Quá trình này giúp giảm thiểu chi phí huấn luyện và tập trung vào việc tối ưu hóa các lớp mới.
- **Thêm các lớp mới:** Để giải quyết bài toán mới, các lớp mới sẽ được thêm vào cuối mô hình pre-trained. Các lớp này sẽ thay thế các lớp fully connected đã bị loại bỏ.
- **Huấn luyện lại (Fine-tuning):** Cuối cùng, mô hình sẽ được huấn luyện lại với dữ liệu của bài toán mới. Quá trình fine-tuning này chỉ cập nhật các lớp mới

và một số lớp cuối cùng của mô hình pre-trained, giúp tối ưu hóa mô hình cho nhiệm vụ cụ thể.

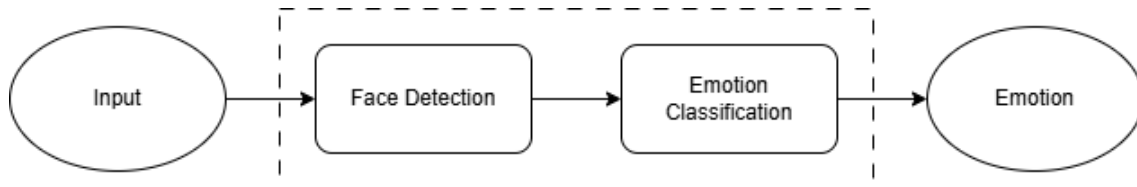
2.4.4 Ưu điểm của Transfer Learning

- **Giảm chi phí tính toán:** Bằng cách sử dụng mô hình đã huấn luyện trước, Transfer Learning giúp tiết kiệm đáng kể tài nguyên tính toán so với việc huấn luyện một mô hình từ đầu.
- **Giải quyết vấn đề kích thước dữ liệu:** Transfer Learning hiệu quả đối với các bài toán có dữ liệu huấn luyện hạn chế, nhờ vào việc tận dụng kiến thức từ các mô hình đã được huấn luyện trên tập dữ liệu lớn.
- **Khả năng tổng quát tốt hơn:** Mô hình Transfer Learning có khả năng tổng quát mạnh mẽ, đặc biệt trong việc xử lý các bài toán với dữ liệu có đặc trưng tương tự với các bài toán đã được huấn luyện trước đó.

CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM

3.1 Tổng quan mô hình

Mô hình nghiên cứu của chúng tôi tuân theo quy trình như được trình bày trong sơ đồ dưới đây:

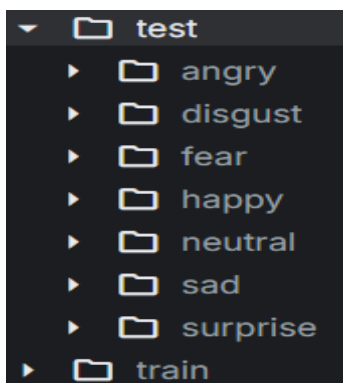


Hình 3.1: Quá trình tổng quan mô hình

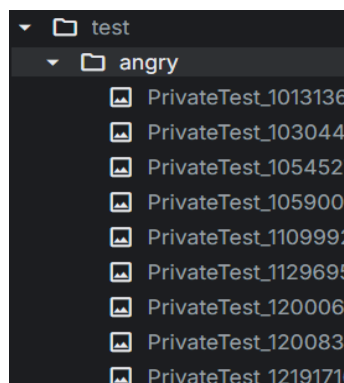
3.2 Dữ liệu sử dụng

Bộ dữ liệu được sử dụng trong nghiên cứu này là FER2013, một bộ dữ liệu nổi tiếng trong lĩnh vực nhận dạng cảm xúc từ khuôn mặt. Bộ dữ liệu này được thu thập và chia sẻ trên Kaggle, chứa hơn 35,000 ảnh khuôn mặt với các biểu cảm khác nhau. Các ảnh này được phân loại thành 7 nhãn cảm xúc chính: tức giận, ghê tởm, sợ hãi, hạnh phúc, buồn bã, ngạc nhiên và trung lập.

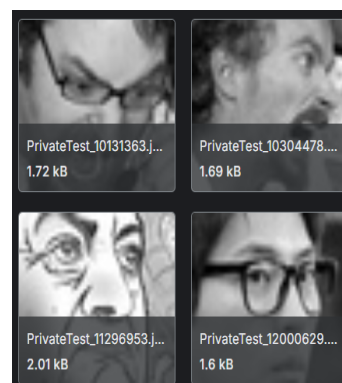
Bộ dữ liệu FER2013 được cấu trúc sẵn thành hai tập dữ liệu chính: tập huấn luyện (train) và tập kiểm tra (test). Mỗi tập dữ liệu bao gồm 7 thư mục, mỗi thư mục tương ứng với một nhãn cảm xúc. Trong mỗi thư mục, các ảnh thuộc về nhãn cảm xúc tương ứng được tổ chức và lưu trữ để phục vụ cho quá trình huấn luyện và kiểm thử mô hình. Dữ liệu này được chuẩn bị sẵn với mục đích giúp việc huấn luyện các mô hình học sâu cho bài toán nhận diện cảm xúc trở nên hiệu quả và dễ dàng hơn.



Hình 3.2: Ảnh từ bộ dữ liệu FER2013



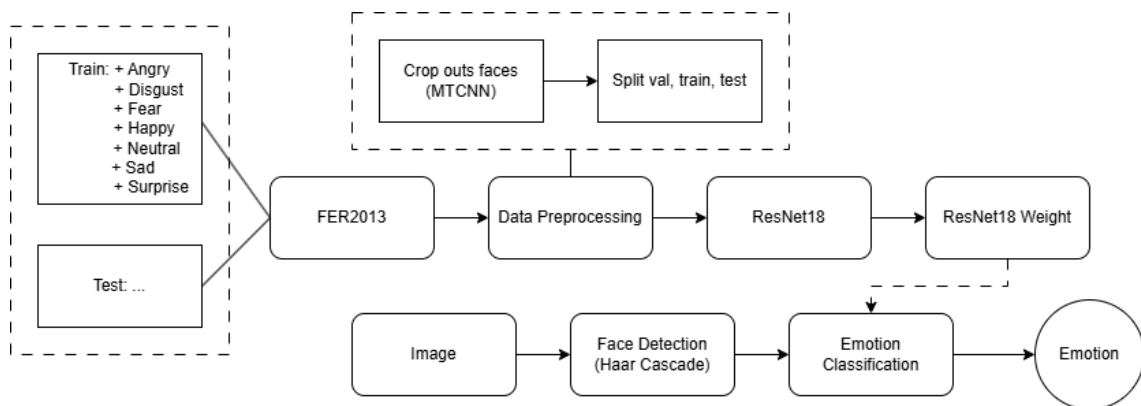
Hình 3.3: Ảnh từ bộ dữ liệu FER2013



Hình 3.4: Ảnh từ bộ dữ liệu FER2013

3.3 Pipeline chi tiết

Quá trình triển khai mô hình được mô tả chi tiết trong pipeline dưới đây:



Hình 3.5: Pipeline dự án

Các quá trình tiền xử lý, đánh giá và triển khai mô hình được thực hiện trên Vscode thông thường; quá trình fine-tune mô hình được thực hiện trên Kaggle sử dụng GPU T4x2

3.4 Tiến hành triển khai thực nghiệm

3.4.1 Tiền xử lý dữ liệu

Các thư viện sử dụng trong bước tiền xử lý dữ liệu bao gồm:

- **os:** Thao tác tạo và sửa các tệp.
- **PIL:** Sử dụng thư viện Image để xử lý ảnh.
- **tqdm:** Theo dõi tiến độ thực hiện.
- **mtcnn:** Sử dụng MTCNN để thực hiện phát hiện khuôn mặt.
- **cv2:** Đọc ảnh từ các đường dẫn tệp.
- **shutil:** Thực hiện sao chép tệp giữa các thư mục.
- **sklearn:** Sử dụng `train_test_split` để chia dữ liệu thành các tập huấn luyện và kiểm tra.

Tiền xử lý dữ liệu đóng vai trò quan trọng trong việc cải thiện hiệu suất của mô hình. Các bước tiền xử lý bao gồm làm sạch dữ liệu, chuẩn hóa hình ảnh và phân chia dữ liệu thành các tập huấn luyện, kiểm tra và xác thực. Quy trình này giúp đảm bảo chất lượng và tính đồng nhất của dữ liệu trước khi đưa vào mô hình học sâu.

Quá trình tiền xử lý bắt đầu với việc sử dụng MTCNN để tách khuôn mặt khỏi ảnh và lưu trữ lại ảnh khuôn mặt đã cắt. Dưới đây là đoạn mã Python thực hiện quá

trình này:

```

1 def crop_and_save_face(image_path, output_path):
2     img = cv2.imread(image_path)
3     img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
4     faces = mtcnn.detect_faces(img_rgb)
5
6     if len(faces) == 0:
7         return False
8
9     bounding_box = faces[0]['box']
10    x, y, w, h = bounding_box
11    x, y = max(0, x), max(0, y)
12
13    cropped_face = img_rgb[y:y+h, x:x+w]
14
15    face_img = Image.fromarray(cropped_face)
16    face_img = face_img.convert('L')
17    face_img = face_img.resize((48, 48))
18    face_img.save(output_path)
19    return True

```

Tiếp theo, đoạn mã dưới đây thực hiện việc tách tập huấn luyện (train) và tập kiểm thử (test), bao gồm việc sao chép ảnh vào các thư mục tương ứng:

```

1 for cls in tqdm(os.listdir(train_root)):
2     cls_path = os.path.join(train_root, cls)
3     images = os.listdir(cls_path)
4
5     train_imgs, val_imgs = train_test_split(images, test_size
6     =0.15, random_state=42)
7
8     os.makedirs(os.path.join(new_train_root, cls), exist_ok=True)
9     os.makedirs(os.path.join(val_root, cls), exist_ok=True)
10
11    for img in train_imgs:
12        shutil.copy2(os.path.join(cls_path, img), os.path.join(
13        new_train_root, cls, img))
14
15    for img in val_imgs:
16        shutil.copy2(os.path.join(cls_path, img), os.path.join(
17        val_root, cls, img))
18
19 for cls in os.listdir(src_test_root):
20     os.makedirs(os.path.join(dst_test_root, cls), exist_ok=True)
21     for img in os.listdir(os.path.join(src_test_root, cls)):

```

```

19         shutil.copy2(
20             os.path.join(src_test_root, cls, img),
21             os.path.join(dst_test_root, cls, img)
22         )

```

3.4.2 Fine-tune trên ResNet18:

Trong quá trình huấn luyện, chúng tôi sử dụng một số thư viện sau để tiền xử lý dữ liệu, huấn luyện và đánh giá mô hình:

- **torch**: Thư viện chính để xây dựng và huấn luyện mô hình học sâu.
- **os**: Thao tác với hệ thống file.
- **numpy**: Thư viện tính toán khoa học.
- **matplotlib**: Thư viện vẽ đồ thị.
- **torchvision**: Thư viện con của PyTorch hỗ trợ các công cụ và mô hình trong thị giác máy tính.
- **datasets, transforms, models** từ **torchvision**: Dùng để tải và biến đổi dữ liệu cũng như sử dụng các mô hình học sâu đã được huấn luyện.
- **DataLoader** từ **torch.utils.data**: Quản lý việc tải và phân phối dữ liệu trong quá trình huấn luyện.
- **CosineAnnealingLR** từ **torch.optim.lr_scheduler**: Dùng để thay đổi học suất theo chu kỳ trong quá trình huấn luyện.

Trước khi đưa dữ liệu vào `DataLoader`, một số phép biến đổi được thực hiện nhằm tăng tính tổng quát của mô hình. Dữ liệu được chuyển về ảnh 3 kênh (RGB) để phù hợp với đầu vào của ResNet18. Các phép biến đổi bao gồm: ngẫu nhiên cắt, thay đổi ánh sáng, affine, lật, xoay và xóa một phần ảnh để mô phỏng các biến đổi trong dữ liệu thực tế.

```

1 transform_train = transforms.Compose([
2     transforms.Grayscale(num_output_channels=3),
3     transforms.RandomResizedCrop(224, scale=(0.8, 1.2)),
4     transforms.RandomApply([transforms.ColorJitter(
5         brightness=0.5, contrast=0.5, saturation=0.5)], p=0.5),
6     transforms.RandomApply([transforms.RandomAffine(0, translate
7         =(0.2, 0.2))], p=0.5),
8     transforms.RandomHorizontalFlip(),
9     transforms.RandomApply([transforms.RandomRotation(10)], p
10        =0.5),
11     transforms.ToTensor(),
12     transforms.Normalize(mean=[0.485], std=[0.229]),

```

```

11     transforms.RandomErasing()
12 ])
13
14 transform_test = transforms.Compose([
15     transforms.Grayscale(num_output_channels=3),
16     transforms.Resize(224),
17     transforms.ToTensor(),
18     transforms.Normalize(mean=[0.485], std=[0.229]),
19 ])

```

Sau khi thực hiện các phép biến đổi trên, dữ liệu được nạp vào DataLoader:

```

1 train_dataset = datasets.ImageFolder(root=f"{dataset_path}/train",
    transform=transform_train)
2 val_dataset = datasets.ImageFolder(root=f"{dataset_path}/val",
    transform=transform_test)
3 test_dataset = datasets.ImageFolder(root=f"{dataset_path}/test",
    transform=transform_test)
4
5 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=
    True)
6 val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False
    )
7 test_loader = DataLoader(test_dataset, batch_size=64, shuffle=
    False)

```

Tiếp theo, cấu hình lại lớp cuối cùng của mô hình ResNet18 để phù hợp với bài toán phân loại cảm xúc, với 7 nhãn cảm xúc đầu ra:

```

1 model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
2
3 num_ftrs = model.fc.in_features
4 model.fc = nn.Sequential(
5     nn.Dropout(0.5),
6     nn.Linear(num_ftrs, 256),
7     nn.ReLU(),
8     nn.Dropout(0.5),
9     nn.Linear(256, 128),
10    nn.ReLU(),
11    nn.Dropout(0.5),
12    nn.Linear(128, 7)
13 )
14
15 device = 'cuda' if torch.cuda.is_available() else 'cpu'
16 model = model.to(device)

```

Cấu hình các tham số huấn luyện, bao gồm số epoch, hàm mất mát, bộ tối ưu hóa và bộ điều chỉnh học suất:

```
1 epochs = 100
2 criterion = nn.CrossEntropyLoss(weight=class_weights_tensor)
3 optimizer = torch.optim.SGD(
4     model.parameters(),
5     lr=0.001,
6     momentum=0.9,
7     weight_decay=0.0001
8 )
9 scheduler = CosineAnnealingLR(optimizer, T_max=100, eta_min=0)
```

Để đánh giá hiệu suất của mô hình trong suốt quá trình huấn luyện, chúng tôi sử dụng hàm đánh giá dưới đây để tính toán accuracy và loss trong mỗi epoch:

```
1 def evaluate_train(model, loader, criterion, device):
2     model.eval()
3     correct = 0
4     total = 0
5     loss_total = 0
6     with torch.no_grad():
7         for inputs, labels in loader:
8             inputs, labels = inputs.to(device), labels.to(device)
9             outputs = model(inputs)
10            loss = criterion(outputs, labels)
11            loss_total += loss.item() * inputs.size(0)
12
13            _, predicted = torch.max(outputs.data, 1)
14
15            correct += (predicted == labels).sum().item()
16            total += labels.size(0)
17
18     loss = loss_total / total
19     acc = correct / total
20
21     return loss, acc
```

Bắt đầu quá trình huấn luyện và lưu trữ các thông số để theo dõi sự thay đổi của accuracy và loss trong quá trình huấn luyện:

```
1 train_losses = []
2 val_losses = []
3 train_accs = []
4 val_accs = []
5 best_val_acc = 0.0
6 epochs_without_improvement = 0
```

```

7
8 for epoch in range(epochs):
9     train_loss = 0.0
10    train_correct = 0
11    total_samples = 0
12
13    model.train()
14    for idx, (inputs, labels) in enumerate(train_loader):
15        inputs, labels = inputs.to(device), labels.to(device)
16
17        optimizer.zero_grad()
18        outputs = model(inputs)
19        loss = criterion(outputs, labels)
20        loss.backward()
21        optimizer.step()
22
23        train_loss += loss.item() * inputs.size(0)
24        train_correct += (torch.argmax(outputs, 1) == labels).sum
25        total_samples += labels.size(0)
26
27    train_loss /= total_samples
28    train_acc = train_correct / total_samples
29    train_losses.append(train_loss)
30    train_accs.append(train_acc)
31
32    val_loss, val_acc = evaluate_train(model, val_loader,
33    criterion, device)
34    val_losses.append(val_loss)
35    val_accs.append(val_acc)
36
37    scheduler.step()
38
39    if val_acc > best_val_acc:
40        best_val_acc = val_acc
41        torch.save(model.state_dict(), '/kaggle/working/
42        best_model.pth')
43        print(f'Best model saved with val acc: {val_acc:.4f}')
44
45    print(f'EPOCH {epoch + 1}/{epochs}: \tTrain loss: {train_loss
46    :.4f} \tVal loss: {val_loss:.4f} \tTrain accuracy: {train_acc:.4
47    f} \tVal accuracy: {val_acc:.4f}')

```

Mô hình tốt nhất sau khi huấn luyện đạt được accuracy là 0.6772. Để theo dõi quá trình học của mô hình, chúng tôi thực hiện việc vẽ đồ thị biểu diễn sự thay đổi

của accuracy và loss qua các epoch:

```
1 fig, ax = plt.subplots(1, 2, figsize=(12, 5))
2
3 ax[0].plot(train_losses, color='blue', label='Train Loss')
4 ax[0].plot(val_losses, color='orange', label='Val Loss')
5 ax[0].set_title('Training and Validation Loss')
6 ax[0].set_xlabel('Epoch')
7 ax[0].set_ylabel('Loss')
8 ax[0].legend()
9 ax[0].grid(True)
10
11 ax[1].plot(train_accs, color='blue', label='Train Accuracy')
12 ax[1].plot(val_accs, color='orange', label='Val Accuracy')
13 ax[1].set_title('Training and Validation Accuracy')
14 ax[1].set_xlabel('Epoch')
15 ax[1].set_ylabel('Accuracy')
16 ax[1].legend()
17 ax[1].grid(True)
18
19 plt.tight_layout()
20 plt.show()
```

Kết quả thu được như sau:



Hình 3.6: Đồ thị Training và Validation Loss/Accuracy

Nhận xét:

Đồ thị Mất Mát (Loss):

- **Train Loss:** Mất mát trên tập huấn luyện giảm đều và ổn định qua các epoch, cho thấy mô hình đang học tốt hơn khi dự đoán trên dữ liệu huấn luyện. Việc giảm ổn định này chỉ ra rằng mô hình đang tối ưu hóa các tham số một cách

hiệu quả.

- **Validation Loss (Màu cam):** Mất mát trên tập kiểm tra ban đầu giảm, nhưng sau epoch thứ 60, sự giảm dần bắt đầu có dao động và sau đó ổn định. Điều này cho thấy trong khi mô hình vẫn cải thiện trên dữ liệu huấn luyện, nó gặp khó khăn khi áp dụng lên tập kiểm tra. Sự ổn định sau epoch 60 có thể chỉ ra rằng mô hình đã đạt đến giới hạn khả năng tổng quát của nó trên tập kiểm tra.

Đồ thị Độ Chính Xác (Accuracy):

- **Train Accuracy:** Độ chính xác trên tập huấn luyện tăng nhanh trong giai đoạn đầu và sau đó ổn định. Điều này cho thấy mô hình học được các đặc trưng trong dữ liệu huấn luyện khá nhanh và có khả năng dự đoán chính xác các nhãn trên dữ liệu huấn luyện.
- **Validation Accuracy:** Độ chính xác trên tập kiểm tra tăng chậm hơn và có một số dao động. Điều này cho thấy mô hình gặp khó khăn trong việc tổng quát hóa trên tập kiểm tra so với tập huấn luyện. Tuy nhiên, sự chênh lệch giữa độ chính xác huấn luyện và kiểm tra nhỏ dần qua các epoch, điều này có thể chỉ ra rằng mô hình bắt đầu overfitting hoặc đã đạt đến giới hạn khả năng tổng quát của nó.

Từ kết quả trên, có thể rút ra một số nhận định: - Mô hình đang học tốt trên tập huấn luyện, nhưng gặp khó khăn trong việc tổng quát hóa trên tập kiểm tra. Việc mất mát không giảm đều trên tập kiểm tra cho thấy mô hình có thể đã bắt đầu overfit, tức là nó quá phù hợp với dữ liệu huấn luyện mà không có khả năng tổng quát tốt lên dữ liệu chưa thấy. - Để cải thiện mô hình, có thể xem xét các phương pháp như điều chỉnh hyperparameters, sử dụng kỹ thuật regularization, hoặc thu thập thêm dữ liệu để giảm thiểu hiện tượng overfitting.

Những nhận xét này sẽ giúp điều chỉnh và tối ưu hóa mô hình trong các lần huấn luyện tiếp theo.

3.4.3 Đánh giá

Thư viện sử dụng:

- **os, sys, numpy, matplotlib, tqdm, seaborn:** Các thư viện hỗ trợ thao tác với tệp, tính toán, vẽ đồ thị, theo dõi tiến độ và hiển thị ma trận nhầm lẫn.
- **from torchvision import datasets, transforms, models:** Cung cấp các công cụ để tải và chuyển đổi dữ liệu hình ảnh, cũng như sử dụng các mô hình học sâu đã được huấn luyện.
- **from torch.utils.data import DataLoader:** Quản lý việc tải và phân phối dữ

liệu trong quá trình huấn luyện.

- **import torch:** Thư viện chính của PyTorch để xây dựng và huấn luyện mô hình.
- **from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix:** *Cchm Ònhgimnhnhprecision, recall, F1 – scorevconfusionmatrix.*

Quá trình đánh giá:

Bắt đầu quá trình đánh giá, chúng tôi lưu trữ các dự đoán và nhãn thực tế vào các danh sách để tính toán accuracy, precision, recall và F1-score. Dưới đây là đoạn mã thực hiện đánh giá trên mô hình:

```

1 all_preds = []
2 all_labels = []
3 correct = 0
4 total_samples = 0
5
6 with torch.no_grad():
7     for inputs, labels in tqdm(ck_loader):
8         inputs, labels = inputs.to(device), labels.to(device)
9         outputs = model(inputs)
10
11         preds = torch.argmax(outputs, 1)
12         all_preds.extend(preds.cpu().numpy())
13         all_labels.extend(labels.cpu().numpy())
14         correct += (preds == labels).sum().item()
15         total_samples += inputs.size(0)
16
17 all_preds = np.array(all_preds)
18 all_labels = np.array(all_labels)
19 accuracy = correct / total_samples

```

Sau đó, chúng tôi tính toán các chỉ số precision, recall và F1-score cho từng lớp cảm xúc và vẽ các biểu đồ để minh họa sự phân phối của các chỉ số này:

```

1 classes = np.arange(len(class_names))
2
3 precision = precision_score(all_labels, all_preds, average=None,
4                             labels=classes)
5 recall = recall_score(all_labels, all_preds, average=None, labels
6                       =classes)
7 f1 = f1_score(all_labels, all_preds, average=None, labels=classes
8              )
9
10 x = np.arange(len(classes))

```



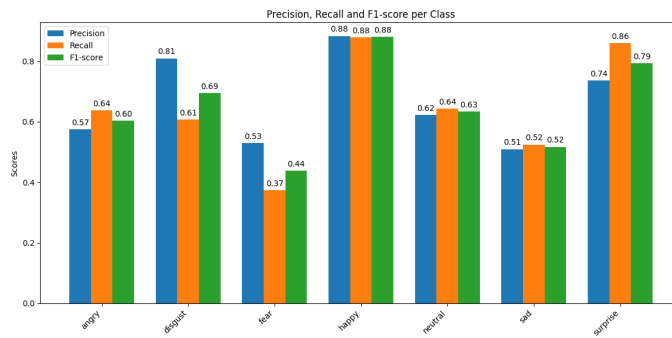
```

8 width = 0.25
9
10 fig, ax = plt.subplots(figsize=(10,6))
11 rects1 = ax.bar(x - width, precision, width, label='Precision')
12 rects2 = ax.bar(x, recall, width, label='Recall')
13 rects3 = ax.bar(x + width, f1, width, label='F1-score')
14
15 ax.set_ylabel('Scores')
16 ax.set_title('Precision, Recall and F1-score per Class')
17 ax.set_xticks(x)
18 ax.set_xticklabels(class_names, rotation=45, ha='right')
19 ax.legend()
20
21 def autolabel(rects):
22     for rect in rects:
23         height = rect.get_height()
24         ax.annotate(f'{height:.2f}',
25                     xy=(rect.get_x() + rect.get_width() / 2,
26                         height),
27                     xytext=(0, 3),
28                     textcoords="offset points",
29                     ha='center', va='bottom')
30
31 autolabel(rects1)
32 autolabel(rects2)
33 autolabel(rects3)
34
35 plt.tight_layout()
36 plt.show()
37
38 cm = confusion_matrix(np.array(all_preds), np.array(all_labels))
39
40 plt.figure(figsize=(8, 6))
41 sns.heatmap(cm, annot=True, fmt='g', cmap='Blues', xticklabels=
42             class_names, yticklabels=class_names)
43 plt.xlabel('Predicted label')
44 plt.ylabel('True label')
45 plt.title('Confusion Matrix')
46 plt.show()

```

Kết quả mô hình được đánh giá trên 3 tập dữ liệu:

- **Tập test của FER2013:** Mô hình đạt accuracy 0.6771 trên tập kiểm tra FER2013. Dưới đây là biểu đồ F1-Score và ma trận nhầm lẫn cho tập dữ liệu FER2013:



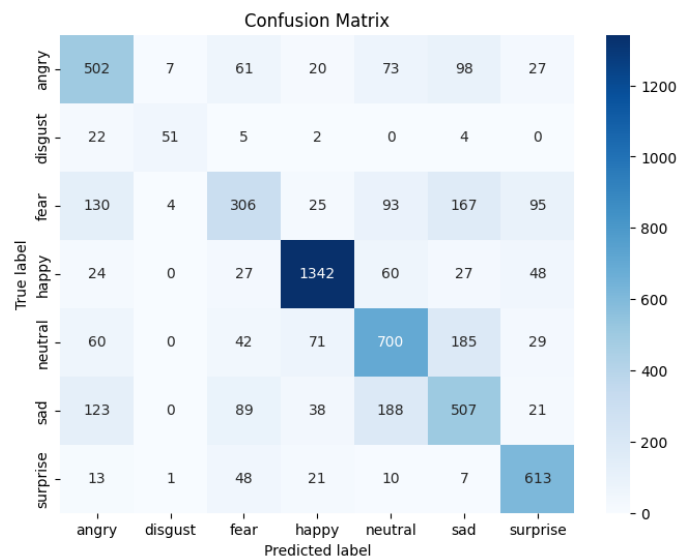
Hình 3.7: Precision, Recall và F1-score trên tập dữ liệu FER2013

Những điểm mạnh:

- Các lớp "Happy" và "Surprise" có độ chính xác (Precision), tỷ lệ thu hồi (Recall) và điểm F1 rất cao, đặc biệt là ở lớp "Happy".
- F1-Score ổn định đối với các lớp như "Neutral" và "Sad".

Các vấn đề cần cải thiện:

- Lớp "Fear" có Precision khá thấp (0.53) và Recall thấp nhất (0.37).
- Lớp "Disgust" có độ chính xác thấp (0.64) và F1-score chưa đạt mức tối ưu (0.60).



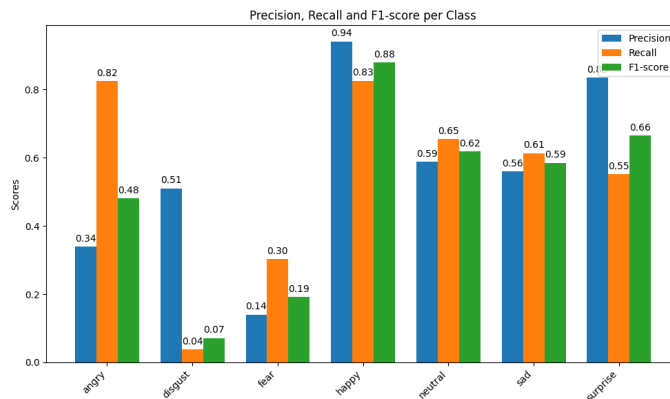
Hình 3.8: Confusion Matrix trên tập dữ liệu FER2013

Nhận xét:

- **Chính xác cao với các cảm xúc phổ biến:** Mô hình thể hiện sự chính xác rất tốt đối với một số cảm xúc như "Happy" và "Surprise." Đây là những lớp mà mô hình có thể phân biệt khá rõ ràng với độ chính xác cao (1.342

mẫu "Happy" được phân loại chính xác, tương đương với giá trị cao nhất trên đường chéo của ma trận).

- **Nhầm lẫn giữa các lớp:** Các cảm xúc "Fear", "Sad", và "Angry" có sự nhầm lẫn khá lớn trong ma trận. Điều này chỉ ra rằng những cảm xúc này có biểu hiện khuôn mặt tương đối giống nhau, làm cho mô hình gặp khó khăn trong việc phân biệt chính xác.
- **Sự nhầm lẫn giữa "Disgust" với các lớp khác:** Mặc dù mô hình phân loại chính xác đối với một số lớp, nhưng cảm xúc "Disgust" vẫn bị nhầm lẫn với các lớp "Angry" và "Sad." Cần cải thiện khả năng phân loại của mô hình cho lớp này.
- **Tập RAF-DB:** Mô hình đạt accuracy 0.66 trên tập RAF-DB. Dưới đây là biểu đồ F1-Score và ma trận nhầm lẫn cho tập dữ liệu RAF-DB:



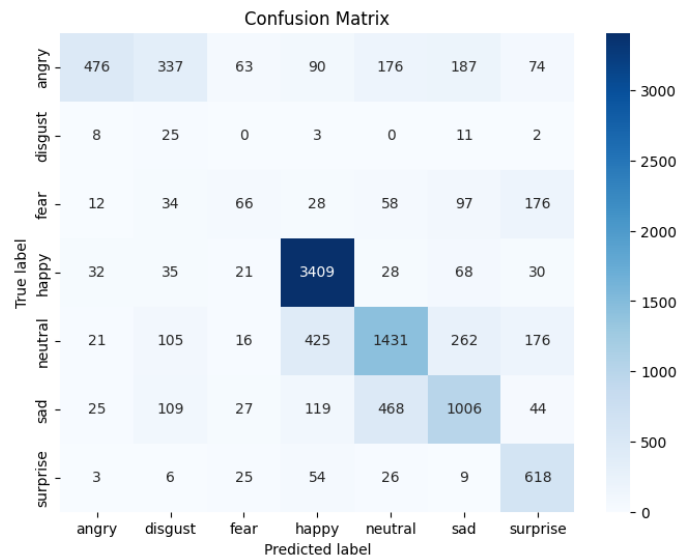
Hình 3.9: Precision, Recall và F1-score trên tập dữ liệu RAF-DB

Điểm mạnh:

- Cảm xúc "Happy" có Precision và Recall rất cao (0.94 và 0.83), cho thấy mô hình có khả năng nhận diện chính xác cảm xúc này.
- Cảm xúc "Surprise" có F1-score khá tốt (0.66).

Điểm cần cải thiện:

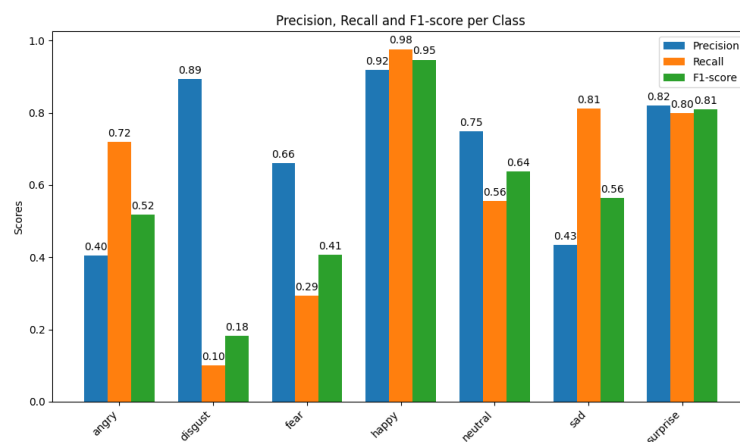
- Cảm xúc "Fear" có Precision và Recall rất thấp (0.30 và 0.19), dẫn đến F1-score thấp (0.14).
- Cảm xúc "Disgust" có Precision và Recall rất thấp (0.04 và 0.07), với F1-score chỉ đạt 0.05.
- Cảm xúc "Angry" có Precision khá cao (0.82), nhưng Recall thấp (0.34), dẫn đến F1-score thấp (0.48).



Hình 3.10: Confusion Matrix trên tập dữ liệu RAF-DB

Nhận xét:

- **Cảm xúc "Happy" và "Surprise":** Các lớp này có tần suất dự đoán đúng cao nhất, đặc biệt là "Happy" với một lượng lớn mẫu được phân loại chính xác.
- **Cảm xúc "Fear":** Có rất nhiều trường hợp "Fear" bị nhầm lẫn với các lớp như "Neutral" và "Sad", cho thấy mô hình cần cải thiện khả năng phân biệt các cảm xúc này.
- **Tập KDEF:** Mô hình đạt accuracy 0.6 trên tập KDEF, một bộ dữ liệu có nhiều góc độ khác nhau của khuôn mặt. Độ chính xác thấp có thể do sự đa dạng về góc độ và vị trí của khuôn mặt trong các bức ảnh.



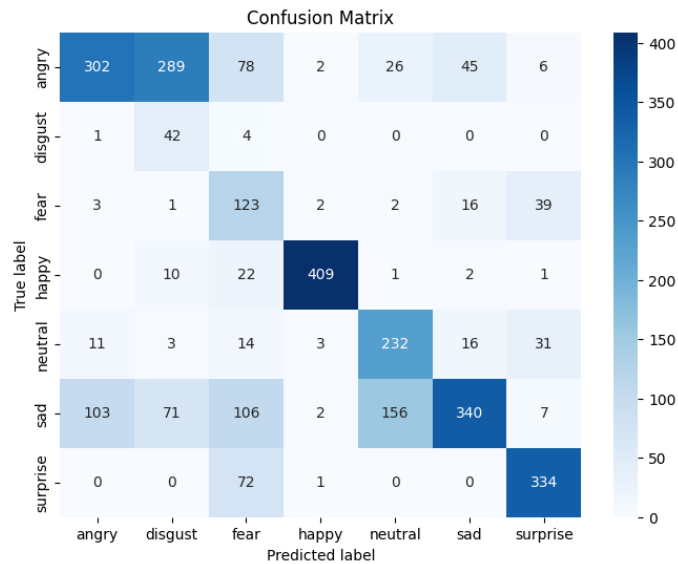
Hình 3.11: Precision, Recall và F1-score trên tập dữ liệu KDEF

Điểm mạnh:

- Cảm xúc "Happy" có Precision và Recall rất cao (0.92 và 0.95).
- Cảm xúc "Surprise" có F1-score tốt (0.81).

Điểm yếu:

- Cảm xúc "Fear" và "Disgust" có độ chính xác thấp, cần cải thiện khả năng phân loại các cảm xúc này.



Hình 3.12: Confusion Matrix trên tập dữ liệu KDEF

Nhận xét:

- **Cảm xúc "Happy" và "Surprise":** Các lớp này có số lượng đúng lớn và ít bị nhầm lẫn với các lớp khác. Điều này cho thấy mô hình khá chính xác trong việc phân loại những cảm xúc dễ phân biệt.
- **Nhầm lẫn giữa "Fear" và "Disgust":** Mô hình gặp phải sự nhầm lẫn lớn giữa các lớp này, cần cải thiện khả năng phân biệt cảm xúc giữa chúng.

Kết luận:

Mô hình đã thể hiện khả năng nhận diện tốt đối với các cảm xúc rõ ràng như "Happy" và "Surprise". Tuy nhiên, đối với các cảm xúc như "Fear", "Disgust", độ chính xác của mô hình còn thấp và cần cải thiện.

3.4.4 Triển khai

Thư viện sử dụng:

- **cv2, numpy, PIL, os, sys, pandas, tqdm, seaborn, torch, torchvision, streamlit:** Các thư viện hỗ trợ đọc và xử lý ảnh, tạo giao diện web, tính toán và vẽ đồ thị, cũng như các công cụ học sâu như PyTorch và các mô hình từ torchvision.

Các hàm cần thiết:

Dưới đây là các hàm được triển khai để phục vụ cho việc nhận diện cảm xúc từ khuôn mặt.

```

1 device = torch.device('cuda' if torch.cuda.is_available() else '
  cpu')
2 classes = ['angry', 'disgust', 'fear', 'happy', 'neutral', 'sad',
  'surprise']
3
4 def load_model(model_path):
5     model = models.resnet18(weights=models.ResNet18_Weights.
  DEFAULT)
6     num_fters = model.fc.in_features
7     model.fc = nn.Sequential(
8         nn.Dropout(0.5),
9         nn.Linear(num_fters, 256),
10        nn.ReLU(),
11        nn.Dropout(0.5),
12        nn.Linear(256, 128),
13        nn.ReLU(),
14        nn.Dropout(0.5),
15        nn.Linear(128, 7)
16    )
17    model.load_state_dict(torch.load(model_path, map_location=
  device))
18    model.to(device)
19    model.eval()
20    return model
21
22 transform = transforms.Compose([
23     transforms.Grayscale(num_output_channels=3),
24     transforms.Resize(224),
25     transforms.ToTensor(),
26     transforms.Normalize(mean=[0.485], std=[0.229]),
27 ])
28
29 def predict(model, face_image: Image.Image):
30     img_tensor = transform(face_image).unsqueeze(0).to(device)
31     with torch.no_grad():
32         outputs = model(img_tensor)
33         probs = torch.nn.Softmax(dim=1)(outputs)
34         predicted_class = torch.argmax(probs, 1).item()
35         predicted_emotion = classes[predicted_class]
36         probabilities = probs.cpu().numpy().flatten()
37     return predicted_emotion, probabilities

```

Triển khai trên 2 phương pháp:**• Nhận ảnh đầu vào từ file tải lên:**

Đoạn mã dưới đây sử dụng thư viện Streamlit để nhận ảnh từ người dùng và thực hiện dự đoán cảm xúc trên khuôn mặt:

```

1      cascade_path = os.path.join(os.path.dirname(__file__)
2      , 'haarcascade_frontalface_default.xml')
3      face_cascade = cv2.CascadeClassifier(cascade_path)
4
5      model_path = os.path.join(os.path.dirname(__file__),
6      '..', 'checkpoint', 'best_model.pth')
7      model = load_model(model_path)
8
9      st.title("Emotion Recognition")
10
11     uploaded_file = st.file_uploader("Choose an Image",
12     type=["jpg", "jpeg", "png"])
13
14     if uploaded_file is not None:
15         image_bytes = uploaded_file.read()
16         try:
17             image = Image.open(io.BytesIO(image_bytes))
18         except UnidentifiedImageError:
19             st.error("The uploaded file is not a valid
20             image. Please upload a valid image file.")
21         else:
22             image_rgb = image.convert('RGB')
23             image_np = np.array(image_rgb)
24             gray = cv2.cvtColor(image_np, cv2.
25             COLOR_RGB2GRAY)
26             faces = face_cascade.detectMultiScale(gray,
27             scaleFactor=1.1, minNeighbors=5)
28
29             if len(faces) > 0:
30                 all_probabilities = []
31                 all_emotions = []
32
33                 for i, (x, y, w, h) in enumerate(faces):
34                     cv2.rectangle(image_np, (x, y), (x +
35                     w, y + h), (0, 255, 0), 2)
36                     face_crop = image_np[y:y + h, x:x + w
37                     ]
38
39                     pil_face = Image.fromarray(face_crop)
40                     predicted_emotion, probabilities =

```

```

predict(model, pil_face)
32         all_probabilities.append(
probabilities)
33         all_emotions.append(predicted_emotion
)
34
35         cv2.putText(image_np,
36                     f'{i + 1}:{
predicted_emotion}-{probabilities[np.argmax(probabilities)
] * 100:.2f}%',
37                     (x, y - 10), cv2.
FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2)
38
39         image_with_boxes = Image.fromarray(
image_np)
40         st.image(image_with_boxes, caption='
Processed Image', use_container_width=True)
41
42         probabilities_df = []
43         for i, emotion in enumerate(all_emotions)
:
44             data = {
45                 'Face Index': f'Face {i + 1}',
46                 **{cls: f'{prob*100:.2f}%' for
cls, prob in zip(classes, all_probabilities[i])}
47             }
48             probabilities_df.append(data)
49
50         result_df = pd.DataFrame(probabilities_df
)
51         st.write("Class Probabilities for
Detected Faces:", result_df)
52         else:
53             st.write("No faces detected in the image.
")
54

```

• Nhận ảnh đầu vào từ webcam:

Đoạn mã dưới đây sử dụng webcam để nhận diện cảm xúc của người dùng theo thời gian thực:

```

1         cascade_path = os.path.join(os.path.dirname(__file__)
, 'haarcascade_frontalface_default.xml')
2         face_cascade = cv2.CascadeClassifier(cascade_path)
3

```



```

4     model_path = os.path.join(os.path.dirname(__file__),
5                               '..', 'checkpoint', 'best_model.pth')
6     model = load_model(model_path)
7
8     def draw_probabilities_bar(frame, probabilities):
9         font = cv2.FONT_HERSHEY_SIMPLEX
10        start_x = frame.shape[1] - 220
11        start_y = 30
12
13        for i, prob in enumerate(probabilities):
14            label = f"{classes[i]}: {prob*100:.2f}%"
15            cv2.putText(frame, label, (start_x, start_y +
16                                    i*30), font, 0.7, (0, 255, 0), 2)
17
18        return frame
19
20    cv2.namedWindow('Emotion Recognition', cv2.
21                    WND_PROP_FULLSCREEN)
22    cv2.setWindowProperty('Emotion Recognition', cv2.
23                            WND_PROP_FULLSCREEN, 1)
24
25    cap = cv2.VideoCapture(0)
26
27    while True:
28        ret, frame = cap.read()
29        if not ret:
30            print("Failed to grab frame")
31            break
32
33        frame = cv2.flip(frame, 1)
34        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
35        faces = face_cascade.detectMultiScale(gray,
36                                              scaleFactor=1.3, minNeighbors=5)
37
38        for (x, y, w, h) in faces:
39            face = frame[y:y+h, x:x+w]
40
41            pil_face = Image.fromarray(cv2.cvtColor(face,
42                                                    cv2.COLOR_BGR2RGB))
43
44            predicted_emotion, probabilities = predict(
45                model, pil_face)
46
47            cv2.rectangle(frame, (x, y), (x+w, y+h), (0,
48                255, 0), 2)

```

```
41         cv2.putText(frame, predicted_emotion, (x, y
42         -10),
43                                     cv2.FONT_HERSHEY_SIMPLEX, 1,
44         (255, 0, 0), 2)
45
46         frame = draw_probabilities_bar(frame,
47         probabilities)
48
49         cv2.imshow('Emotion Recognition', frame)
50
51         if cv2.waitKey(1) & 0xFF == ord('q'):
52             break
53
54     cap.release()
55     cv2.destroyAllWindows()
```

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết luận

Đồ án này đã nghiên cứu và phát triển một mô hình học sâu để nhận diện và phân loại các cảm xúc từ khuôn mặt. Mô hình được xây dựng dựa trên các kiến trúc mạng nơ-ron hiện đại như ResNet18 và MTCNN, áp dụng cho bộ dữ liệu FER2013, RAF-DB và KDEF. Các bước tiền xử lý dữ liệu, bao gồm phát hiện khuôn mặt bằng MTCNN và biến đổi hình ảnh, đã được triển khai để nâng cao hiệu quả huấn luyện và phân loại.

Qua quá trình huấn luyện và đánh giá, mô hình đã đạt được kết quả khả quan, đặc biệt là đối với các lớp cảm xúc phổ biến như "Happy" và "Surprise". Tuy nhiên, một số lớp như "Fear", "Disgust" vẫn còn gặp khó khăn trong việc nhận diện chính xác. Các chỉ số Precision, Recall và F1-Score của các lớp này cần được cải thiện để nâng cao hiệu suất tổng thể của mô hình.

Đồ án đã giải quyết thành công các vấn đề liên quan đến việc xây dựng và huấn luyện mô hình nhận diện cảm xúc, nhưng vẫn tồn tại một số vấn đề như hiện tượng overfitting đối với các lớp ít xuất hiện và cần thêm dữ liệu huấn luyện để cải thiện.

4.2 Hướng phát triển trong tương lai

Trong tương lai, để nâng cao hiệu quả của mô hình, có thể thực hiện một số cải tiến sau:

- **Thu thập thêm dữ liệu huấn luyện:** Các lớp ít gặp như "Fear" và "Disgust" cần có thêm dữ liệu huấn luyện để giúp mô hình học được các đặc trưng của các cảm xúc này. Dữ liệu từ các nguồn khác nhau và với nhiều góc độ khuôn mặt sẽ giúp cải thiện khả năng tổng quát của mô hình.
- **Sử dụng các kỹ thuật regularization và data augmentation:** Việc áp dụng các kỹ thuật như dropout, batch normalization và data augmentation có thể giúp giảm thiểu overfitting và cải thiện độ chính xác của mô hình. Bên cạnh đó, các phương pháp tổng hợp dữ liệu từ nhiều bộ dữ liệu khác nhau sẽ giúp mô hình học được các đặc điểm phong phú hơn.
- **Cải thiện các mô hình tiên tiến:** Các mô hình tiên tiến như MTCNN có thể được cải tiến thêm để tối ưu hóa độ chính xác trong việc phát hiện khuôn mặt và căn chỉnh các đặc điểm khuôn mặt. Các phương pháp tăng cường độ phân giải ảnh, ví dụ như sử dụng SRCNN, EDSR hoặc SRGAN, có thể giúp cải thiện chất lượng ảnh đầu vào và tăng độ chính xác của mô hình.

- **Áp dụng các kiến trúc mạng sâu khác:** Các kiến trúc mạng sâu như DenseNet, EfficientNet hay các mô hình học sâu mới nhất có thể được thử nghiệm để so sánh hiệu quả với ResNet và cải thiện kết quả phân loại. Điều này có thể mang lại các cải thiện về tính tổng quát và khả năng phân loại chính xác hơn.
- **Tích hợp với các hệ thống thực tế:** Đưa mô hình vào các hệ thống nhận diện cảm xúc thực tế, như trong các ứng dụng hỗ trợ khách hàng, phân tích cảm xúc người dùng, hay trong các hệ thống giám sát cảm xúc trong học tập và công việc. Phát triển hệ thống nhận diện cảm xúc thông qua không chỉ khuôn mặt mà còn qua các yếu tố như cử chỉ và ngôn ngữ cơ thể cũng sẽ là một hướng đi quan trọng.
- **Khám phá cảm xúc đa phương thức:** Mô hình hiện tại chỉ dừng lại ở việc nhận diện cảm xúc qua khuôn mặt. Tuy nhiên, cảm xúc là một đề tài phức tạp, không chỉ thể hiện qua khuôn mặt mà còn qua cử chỉ và ngôn ngữ cơ thể. Việc phát triển các hệ thống như Caer-Net, với hai luồng đầu vào – một luồng xử lý khuôn mặt (face stream) và một luồng xử lý bối cảnh chung quanh (context stream) – có thể giúp mô hình trở nên toàn diện và chính xác hơn trong việc nhận diện cảm xúc.

Với những cải tiến và phát triển tiếp theo, mô hình sẽ có khả năng nhận diện cảm xúc chính xác hơn và có thể ứng dụng rộng rãi hơn trong các lĩnh vực như giáo dục, chăm sóc sức khỏe và công nghệ giải trí.